

Đồ án Kỹ thuật Lập trình – Chủ đề 7: Từ điển Anh-Việt (OOP)

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO ĐỒ ÁN
MÔN: KỸ THUẬT LẬP TRÌNH

Chủ đề 7:

TỪ ĐIỂN ANH – VIỆT

(Phiên bản nâng cấp hướng đối tượng)

Sinh viên thực hiện: Phạm Xuân Việt - Lê Mạnh Cường

MSSV: [Mã số sinh viên]

Lớp: 169313

Giảng viên hướng dẫn: [Tên giảng viên]

Hà Nội, 2026

Contents

1	GIỚI THIỆU ĐỀ TÀI	3
1.1	Mô tả đề tài	3
1.2	Yêu cầu chức năng	3
1.3	Yêu cầu phi chức năng	3
2	PHÂN TÍCH YÊU CẦU	4
2.1	Phân tích chức năng theo nhóm trách nhiệm	4
2.2	Nhận xét	4
3	THIẾT KẾ HƯỚNG ĐỐI TƯỢNG	5
3.1	Các nguyên tắc OOP đã áp dụng	5
3.1.1	Đóng gói (Encapsulation)	5
3.1.2	Kế thừa (Inheritance)	5
3.1.3	Đa hình (Polymorphism)	5
3.1.4	Trừu tượng hoá (Abstraction)	6
3.2	Hệ thống lớp – sơ đồ tổng quan	6
3.3	Các mẫu thiết kế (Design Patterns)	6
3.3.1	Strategy Pattern – Chiến lược tìm kiếm	6
3.3.2	Facade Pattern – Lớp Application	6
3.3.3	Template Method ngầm – IPersistable	7
3.4	Hệ thống Exception	7
4	TRIỂN KHAI CHƯƠNG TRÌNH	8
4.1	Cấu trúc thư mục	8
4.2	Mô tả chi tiết các lớp	8
4.2.1	Lớp template <code>DynamicArray<T></code>	8
4.2.2	Lớp <code>Word</code>	8
4.2.3	Strategy Pattern: <code>ISearchStrategy</code>	9
4.2.4	Lớp <code>Dictionary</code>	9
4.2.5	Lớp <code>History</code> (Linked List + Rule of Three)	9
4.2.6	Facade: <code>Application</code>	9
4.3	Định dạng file dữ liệu	10
5	KIỂM THỬ	11
6	TỔNG KẾT CÁC KỸ THUẬT ĐÃ VẬN DỤNG	13
6.1	Kỹ thuật về thiết kế hướng đối tượng (OOP)	13
6.2	Các mẫu thiết kế (Design Patterns)	13
6.3	Kỹ thuật về thuật toán và cấu trúc dữ liệu	13

6.4	So sánh với phiên bản trước nâng cấp	14
6.5	Kỹ thuật kiểm thử	14
A	MÃ NGUỒN CÁC THÀNH PHẦN CHÍNH	15
A.1	Hàm main	15
A.2	Facade: Application	16
A.2.1	Application.h	16
A.2.2	Application.cpp	17
A.3	Hệ thống Exception	18
A.4	Interface IPersistable	19
A.5	Strategy Pattern – ISearchStrategy	20
A.6	Dictionary.cpp (trích)	20
A.7	History.cpp (Rule of Three)	22

Chương 1. GIỚI THIỆU ĐỀ TÀI

1.1. Mô tả đề tài

Đề tài “Từ điển Anh – Việt” thuộc Chủ đề 7 của môn Kỹ thuật Lập trình. Mục tiêu xây dựng chương trình tra cứu từ vựng với các tính năng cốt lõi của một từ điển điện tử: quản lý từ vựng, tra cứu chính xác, tra cứu gần đúng dựa trên thuật toán so khớp chuỗi, lưu lịch sử và đánh dấu từ yêu thích.

Phiên bản trình bày trong báo cáo này là phiên bản nâng cấp theo hướng đối tượng (OOP) của một thiết kế thủ tục – đối tượng đơn giản trước đó. Việc nâng cấp nhằm minh họa rõ nét các nguyên tắc và mẫu thiết kế hướng đối tượng đã học, đồng thời cải thiện độ mở rộng và bảo trì của hệ thống.

1.2. Yêu cầu chức năng

- Quản lý từ vựng: thêm, sửa, xóa từ. Mỗi từ bao gồm nghĩa tiếng Việt, ví dụ, từ đồng nghĩa.
- Tra cứu chính xác (không phân biệt hoa - thường).
- Tra cứu gần đúng dựa trên khoảng cách Levenshtein, gợi ý các từ tương tự khi người dùng gõ sai.
- Lịch sử tra cứu (most-recently-used, có giới hạn).
- Đánh dấu từ vựng yêu thích.
- Lưu trữ dữ liệu trong file text.

1.3. Yêu cầu phi chức năng

Đề bài yêu cầu không sử dụng các cấu trúc dữ liệu nâng cao hay thư viện có sẵn (`std::vector`, `std::list`, `std::map`, `std::sort`, ...). Ngoài ra, phiên bản OOP còn đặt thêm các yêu cầu nội bộ:

- Áp dụng đầy đủ 4 trụ cột OOP: đóng gói, kế thừa, đa hình, trừu tượng hoá.
- Áp dụng ít nhất 2 mẫu thiết kế (design pattern) đã học.
- Tách biệt rõ tầng dữ liệu, tầng thuật toán, tầng giao diện.
- Sử dụng exception thay cho mã trả về `bool/int` để báo lỗi.

Chương 2. PHÂN TÍCH YÊU CẦU

2.1. Phân tích chức năng theo nhóm trách nhiệm

Trên cơ sở các yêu cầu chức năng, hệ thống được phân rã thành các nhóm trách nhiệm. Mỗi nhóm sau này sẽ tương ứng với một (hoặc một số) lớp:

Nhóm trách nhiệm	Mô tả
Mô hình dữ liệu	Biểu diễn 1 từ vựng (Word) với các trường: từ tiếng Anh, danh sách nghĩa, ví dụ, đồng nghĩa.
Quản lý kho từ	Tập hợp các từ (Dictionary), hỗ trợ thêm/sửa/xóa và truy vấn.
Thuật toán tìm kiếm	Các chiến lược khác nhau: chính xác, gần đúng. Phải có khả năng đổi chiến lược runtime và dễ mở rộng.
Trạng thái phụ trợ	Lịch sử tra cứu (FIFO có giới hạn), danh sách yêu thích.
Bền vững hoá	Lưu/tải dữ liệu xuống/từ file text. Áp dụng cho Dictionary , History , Favorites – cùng một giao diện.
Giao diện người dùng	Hiển thị menu, nhận lệnh, hiển thị kết quả. Phải tách biệt với phần logic để có thể thay thế bằng GUI sau này.
Điều phối	Khởi tạo, kết nối các thành phần, quản lý vòng đời.

2.2. Nhận xét

Phân tích trên cho thấy ba quan sát quan trọng:

- Có 3 nhóm lớp cùng cần khả năng lưu/tải file (**Dictionary**, **History**, **Favorites**)
⇒ nên trừu tượng hoá thành một interface **IPersistable** chung.
- Hệ thống có nhiều thuật toán tìm kiếm khác nhau ⇒ áp dụng **Strategy Pattern**, gói mỗi thuật toán thành một lớp con của **ISearchStrategy**.
- Giao diện người dùng cần dễ thay thế ⇒ trừu tượng hoá thành interface **IUserInterface**, hiện tại có **ConsoleUI**, tương lai có thể có **GuiUI**.

Chương 3. THIẾT KẾ HƯỚNG ĐỐI TƯỢNG

3.1. Các nguyên tắc OOP đã áp dụng

3.1.1. Đóng gói (*Encapsulation*)

Tất cả các lớp đều có trường dữ liệu `private`, được truy cập thông qua getter/setter công khai. Ví dụ lớp `Word`:

```
1 class Word {
2 private:
3     std::string english_;
4     DynamicArray<std::string> meanings_;
5     DynamicArray<std::string> examples_;
6     DynamicArray<std::string> synonyms_;
7
8 public:
9     const std::string& getEnglish() const { return english_; }
10    void addMeaning(const std::string& m) {
11        meanings_.push_back(m); }
12    // ...
13};
```

Lưu ý quy ước đặt tên: trường `private` có hậu tố underscore (`english_`, `meanings_`) – một quy ước phổ biến trong các dự án C++ chuyên nghiệp (Google Style Guide).

3.1.2. Kế thừa (*Inheritance*)

Có ba cây kế thừa chính trong hệ thống:

- `IPersistable` (interface) ← `Dictionary`, `History`, `Favorites`
- `ISearchStrategy` (interface) ← `ExactSearch`, `ApproximateSearch`
- `IUserInterface` (interface) ← `ConsoleUI`
- `std::exception` ← `DictionaryException` ← `WordNotFoundException`, `DuplicateWordException`, `FileIOException`, `InvalidInputException`

3.1.3. Đa hình (*Polymorphism*)

Đa hình động được sử dụng tại nhiều điểm:

- `Dictionary::search` nhận một `const ISearchStrategy&` – cùng một lời gọi `search(strategy, query, results)` có thể thực thi thuật toán khác nhau tùy vào kiểu thực tế của `strategy`.
- `Application` giữ con trỏ `IUserInterface*` – sau này thay `ConsoleUI` bằng `GuiUI` chỉ cần đổi một dòng `new ConsoleUI(...)` thành `new GuiUI(...)`.
- Catch `DictionaryException` bắt được tất cả exception con nhờ đa hình của hàm `what()`.

3.1.4. Trừu tượng hoá (Abstraction)

Các interface (`IPersistable`, `ISearchStrategy`, `IUserInterface`) đều là abstract class chứa pure virtual function. Client code không cần biết chi tiết cài đặt – chỉ làm việc thông qua interface.

3.2. Hệ thống lớp – sơ đồ tổng quan

Sơ đồ lớp dưới đây thể hiện toàn bộ kiến trúc của hệ thống. Các ô màu vàng nhạt là interface, các ô màu xanh dương/xanh lá/hồng/tím là các lớp cụ thể, các mũi tên rỗng là quan hệ kế thừa, các mũi tên đứt nét là quan hệ phụ thuộc.

[Hình 3.1. Sơ đồ lớp tổng quan của hệ thống]
(Chèn file docs/class_diagram.png vào đây)

Figure 3.1: Sơ đồ lớp tổng quan của hệ thống

3.3. Các mẫu thiết kế (Design Patterns)

3.3.1. Strategy Pattern – Chiến lược tìm kiếm

Strategy Pattern đóng gói các thuật toán khác nhau vào các lớp riêng và cho phép thay đổi thuật toán tại thời điểm chạy. Trong hệ thống này:

- Interface `ISearchStrategy` định nghĩa hợp đồng `search(words, query, results)`.
- `ExactSearch` cài đặt thuật toán tìm chính xác (case-insensitive).
- `ApproximateSearch` cài đặt thuật toán tìm gần đúng (Levenshtein).
- `Dictionary::search` nhận một `const ISearchStrategy&` – không phụ thuộc vào thuật toán cụ thể.

Lợi ích thực tế: trong tác vụ tra cứu, `ConsoleUI` thử `ExactSearch` trước, nếu không có kết quả thì chuyển sang `ApproximateSearch` – đổi chiến lược động ngay trong một lần thao tác. Khi muốn thêm thuật toán mới (`PrefixSearch` dùng Trie, `PhoneticSearch` dùng Soundex, ...) chỉ cần tạo lớp mới kế thừa `ISearchStrategy` mà không sửa `Dictionary`.

3.3.2. Facade Pattern – Lớp Application

Facade Pattern cung cấp một giao diện thống nhất cho một tập các giao diện trong một hệ thống con. `Application` gói tất cả các thành phần (`Dictionary`, `History`, `Favorites`, `IUserInterface`) và expose ba phương thức đơn giản: `initialize()`, `run()`, `shutdown()`.

Nhờ vậy `main()` chỉ còn khoảng 15 dòng:

```

1  int main() {
2      try {
3          Application app;
4          app.initialize();
5          app.run();
6          app.shutdown();
7      } catch (const DictionaryException& e) {
8          std::cerr << "Loi: " << e.what() << "\n";
9          return 1;
10     }
11     return 0;
12 }

```

3.3.3. Template Method ngầm – IPersistable

IPersistable không phải Template Method điển hình, nhưng chia sẻ tinh thần tương tự: ba lớp con (Dictionary, History, Favorites) đều cài đặt cùng cặp phương thức `saveToFile/loadFromFile` theo cách phù hợp với cấu trúc dữ liệu nội bộ của mình. Client code có thể duyệt một mảng `IPersistable*` và gọi `saveToFile()` đồng loạt mà không cần biết kiểu cụ thể.

3.4. Hệ thống Exception

Thay vì trả về `bool` hoặc `-1` để báo lỗi, hệ thống dùng exception. Cây kế thừa:

```

std::exception
  DictionaryException      (base, chứa message_)
    WordNotFoundException
    DuplicateWordException
    FileIOException
    InvalidInputException

```

Mọi exception trong hệ thống đều xuất phát từ `DictionaryException`, nên client có thể bắt một lần là đủ:

```

1  try {
2      dict_.addWord(w);
3      std::cout << "Da them tu.\n";
4  } catch (const DuplicateWordException& e) {
5      std::cout << "Loi: " << e.what() << "\n";
6  }

```

Ưu điểm: tách rõ luồng thành công và luồng lỗi, code chính dễ đọc hơn, không phải kiểm tra trị trả về sau mỗi lệnh.

Chương 4. TRIỂN KHAI CHƯƠNG TRÌNH

4.1. Cấu trúc thư mục

```
TuDienAnhViet/  
src/  
    DynamicArray.h           Template mảng động  
    Exceptions.h             Hierarchy exception  
    StringUtils.h / .cpp     Utility class  
    IPersistable.h           Interface persistence  
    IUserInterface.h         Interface UI  
    ISearchStrategy.h        Interface chiến lược tìm kiếm  
    Word.h / .cpp            Model: 1 từ vựng  
    ExactSearch.h / .cpp     Strategy: tìm chính xác  
    ApproximateSearch.h / .cpp Strategy: tìm gần đúng  
    Dictionary.h / .cpp      Repository: kho từ  
    History.h / .cpp         Lịch sử (linked list)  
    Favorites.h / .cpp       Yêu thích  
    ConsoleUI.h / .cpp       Giao diện console  
    Application.h / .cpp     Facade  
    main.cpp  
data/  
    dictionary.txt  
    history.txt  
    favorites.txt  
docs/  
    class_diagram.png  
Makefile  
build.bat  
README.md
```

4.2. Mô tả chi tiết các lớp

4.2.1. Lớp template *DynamicArray<T>*

Cài đặt mảng động kiểu generic, đầy đủ Rule of Three (copy constructor, copy assignment, destructor). Cơ chế nhân đôi capacity khi đầy cho `push_back` đạt $O(1)$ amortized.

4.2.2. Lớp *Word*

Đại diện một từ với 4 thuộc tính private. Cài đặt `operator==`, `operator!=` (so sánh theo english không phân biệt hoa – thường) và `friend operator<<` (in ra ostream). Việc dùng `operator<<` thay cho hàm `print()` có ưu điểm: cú pháp tự nhiên, có thể ghép nối (`std::cout << word1 << word2`), tương thích với mọi `std::ostream` (cả console lẫn file).

4.2.3. Strategy Pattern: ISearchStrategy

Đoạn code thể hiện rõ Strategy Pattern trong `ConsoleUI::handleLookup`:

```

1 // Bước 1: dùng Strategy ExactSearch
2 DynamicArray<SearchResult> exactResults;
3 dict_.search(exactSearch_, q, exactResults);
4
5 if (!exactResults.empty()) {
6     const Word& w = dict_.at(exactResults[0].wordIndex);
7     std::cout << w;
8     history_.add(w.getEnglish());
9     return;
10 }
11
12 // Bước 2: chuyển sang Strategy ApproximateSearch
13 DynamicArray<SearchResult> approxResults;
14 dict_.search(approxSearch_, q, approxResults);

```

Cùng một phương thức `dict_.search()` được gọi với hai strategy khác nhau và cho hai hành vi khác nhau – đây chính là điểm mạnh của đa hình.

4.2.4. Lớp Dictionary

Quản lý `DynamicArray<Word>`. Cung cấp các phép CRUD ném exception khi gặp lỗi:

```

1 void Dictionary::addWord(const Word& w) {
2     if (findIndexExact(w.getEnglish()) != -1) {
3         throw DuplicateWordException(w.getEnglish());
4     }
5     words_.push_back(w);
6 }
7
8 void Dictionary::deleteWord(const std::string& english) {
9     int idx = findIndexExact(english);
10    if (idx == -1) {
11        throw WordNotFoundException(english);
12    }
13    words_.removeAt(idx);
14 }

```

4.2.5. Lớp History (Linked List + Rule of Three)

History dùng danh sách liên kết đơn, quản lý heap thông qua `new/delete`. Vì có pointer member nên phải cài đặt đầy đủ Rule of Three: copy constructor, copy assignment, destructor. Hàm `copyFrom()` thực hiện deep copy theo đúng thứ tự để giữ nguyên ngữ nghĩa “most-recently-used”.

4.2.6. Facade: Application

```

1 void Application::initialize() {
2     try {
3         dict_.loadFromFile(DICT_FILE);
4     } catch (const FileIOException&) {
5         // OK: là n đã u chạy, chưa có filetry
        history.loadFromFile(HIST_FILE); catch(const FileIOException) try favorites.loadFromFile(FAV_FILE);
        ctthaybngGuiUIsaunyuui=new ConsoleUI(dict,history,favorites);
    }
}

```

4.3. Định dạng file dữ liệu

File dictionary.txt giữ nguyên định dạng của phiên bản trước để tương thích ngược:

```

@english=hello
meaning=xin chào|chào hỏi
example=Hello, how are you?|Say hello to your mom.
synonym=hi|hey|greetings
@end

```

history.txt và favorites.txt mỗi dòng một từ tiếng Anh.

Chương 5. KIỂM THỬ

Các tình huống kiểm thử được thực hiện trên file dữ liệu mẫu gồm 20 từ. Đặc biệt chú trọng các test case kiểm tra cơ chế OOP: exception, polymorphism, strategy.

STT	Tình huống / Cơ chế kiểm tra	Thao tác	Kết quả mong đợi	KQ	
1	ExactSearch hoạt động	strategy	Chọn 1, nhập hello	Hiển thị đủ nghĩa, ví dụ, đồng nghĩa	Đạt
2	Operator« của Word		Tra một từ	Output đúng format định nghĩa trong operator«	Đạt
3	Polymorphism: Strategy runtime	chuyển	Chọn 1, nhập computer (sai)	ExactSearch fail → tự động chuyển sang ApproximateSearch, gợi ý computer khoảng cách 1	Đạt
4	ApproximateSearch từ sai nặng	với	Nhập beatiful	Gợi ý beautiful khoảng cách 1	Đạt
5	Encapsulation: thể truy cập trực tiếp dict_.words_	không	Test compile: viết code dict_.words_ trong main	Compile lỗi private member	Đạt
6	DuplicateWordException		Thêm từ hello đã tồn tại	Throw DuplicateWordException, message rõ ràng, chương trình không crash	Đạt
7	WordNotFoundException		Sửa từ không tồn tại	Throw WordNotFoundException, được catch và in lỗi gọn	Đạt
8	Đa hình bắt lỗi qua base class		main() bắt const DictionaryException&	Bắt được mọi exception con	Đạt
9	IPersistable polymorphism		Chọn lưu: gọi saveToFile() trên cả 3 đối tượng	Cả 3 file được lưu, mỗi loại theo định dạng riêng	Đạt
10	History – linked list MRU		Tra 5 từ, sau đó tra lại từ đầu tiên	Từ đó nhảy lên đầu, không bị nhân đôi	Đạt
11	History vượt giới hạn		Tra 25 từ khác nhau	Chỉ giữ 20 từ gần nhất (maxSize)	Đạt

STT	Tình huống / Cơ chế kiểm tra	Thao tác	Kết quả mong đợi	KQ
12	Rule of Three – copy History	Tạo bản sao History h2 = h1, sửa h2, kiểm tra h1	h1 không bị ảnh hưởng (deep copy)	Đạt
13	Favorites – encapsulation kiểm soát thêm/xóa	Thêm <code>study</code> vào yêu thích, xóa <code>study</code> khỏi từ điển	<code>study</code> tự động biến mất khỏi yêu thích	Đạt
14	Lưu/tải – bền vững hoá	Thay đổi, thoát chương trình, mở lại	Mọi thay đổi được giữ nguyên	Đạt
15	Const correctness	Cố gọi <code>addWord()</code> trên <code>const Dictionary</code>	Compile báo lỗi – tham chiếu <code>const</code> không thể gọi <code>non-const method</code>	Đạt
16	RAII – giải phóng tài nguyên	Thoát chương trình, kiểm tra <code>valgrind</code>	Không rò rỉ bộ nhớ	Đạt
17	Tra chuỗi rỗng / menu sai	Nhập rỗng, chọn 99	Báo lỗi gọn, không crash	Đạt

Ghi chú: hình ảnh chụp màn hình thực tế của từng test case sẽ được chèn vào báo cáo in để minh hoạ.

Chương 6. TỔNG KẾT CÁC KỸ THUẬT ĐÃ VẬN DỤNG

6.1. Kỹ thuật về thiết kế hướng đối tượng (OOP)

Kỹ thuật	Vận dụng tại
Encapsulation	Tất cả các lớp: trường <code>private</code> + getter/setter công khai. Quy ước hậu tố <code>underscore</code> .
Inheritance	3 cây kế thừa: <code>IPersistable</code> , <code>ISearchStrategy</code> , <code>IUserInterface</code> ; cộng thêm hệ thống <code>Exception</code> .
Polymorphism	Pure virtual function trong các interface. <code>Dictionary::search</code> nhận <code>const ISearchStrategy&</code> , <code>Application</code> giữ <code>IUserInterface*</code> .
Abstraction	Các interface chỉ định nghĩa hợp đồng, ẩn cài đặt cụ thể.
Operator overloading	<code>Word::operator==</code> , <code>friend operator<<</code>
Template (generic)	<code>DynamicArray<T></code> dùng được cho mọi kiểu (<code>string</code> , <code>Word</code> , <code>SearchResult</code> , ...)
Const correctness	Mọi getter, mọi tham chiếu read-only đều là <code>const</code> . Đảm bảo invariant của object.
Rule of Three	<code>History</code> (linked list quản lý heap): copy ctor + copy assign + dtor đầy đủ.
RAII	Mọi tài nguyên cấp phát đều giải phóng trong destructor: <code>DynamicArray</code> , <code>History::Node</code> , bảng DP trong <code>Levenshtein</code> .
Exception handling	Thay return code bằng exception, cây kế thừa từ <code>std::exception</code> , polymorphism trong <code>catch</code> .

6.2. Các mẫu thiết kế (Design Patterns)

- **Strategy Pattern:** `ISearchStrategy` + `ExactSearch`, `ApproximateSearch`.
- **Facade Pattern:** `Application` gói toàn bộ hệ thống con.
- **Dependency Injection:** `ConsoleUI` và `Application` nhận tham chiếu `Dictionary/History/...` thay vì tự tạo.
- **Single Responsibility Principle:** mỗi lớp một trách nhiệm. UI / data / algorithm tách 3 layer.

6.3. Kỹ thuật về thuật toán và cấu trúc dữ liệu

- Quy hoạch động: `Levenshtein distance` với bảng DP 2 chiều tự cấp phát.

- Quicksort tự cài cho `SearchResult`.
- Mảng động (template) – thay `vector`.
- Danh sách liên kết đơn với FIFO bounded – thay `deque`.
- Pruning sớm theo chênh lệch độ dài trong `ApproximateSearch`.

6.4. So sánh với phiên bản trước nâng cấp

Khía cạnh	Phiên bản trước	Phiên bản OOP
Word	<code>struct</code> , trường <code>public</code>	<code>class</code> , <code>private</code> + <code>getter/setter</code> + <code>operator</code>
Tìm kiếm	Hai hàm riêng <code>findExact</code> / <code>findApproximate</code>	Strategy Pattern – đổi run-time
Lưu/tải	Namespace <code>FileIO</code> với <code>free</code> function	Interface <code>IPersistable</code> , các lớp tự lo
Báo lỗi	Trả về <code>bool</code> / <code>-1</code>	Exception hierarchy
UI	Code menu nằm trong <code>main.cpp</code>	Lớp <code>ConsoleUI</code> tách biệt
<code>main()</code>	~270 dòng	~20 dòng (Facade)
Mở rộng UI	Phải viết lại <code>main</code>	Chỉ thêm lớp mới implement <code>IUserInterface</code>
Mở rộng tìm kiếm	Phải sửa <code>Dictionary</code>	Chỉ thêm lớp mới implement <code>ISearchStrategy</code>

6.5. Kỹ thuật kiểm thử

- Phân nhóm test case theo cơ chế OOP cần kiểm tra (exception, polymorphism, encapsulation, ...).
- Test biên: chuỗi rỗng, lựa chọn ngoài phạm vi, vượt giới hạn lịch sử.
- Test tích hợp: thoát – mở lại để kiểm tra tính toàn vẹn dữ liệu.
- Test ngữ pháp: thử biên dịch các đoạn code vi phạm `const/private` để xác nhận encapsulation thực sự hoạt động.

Chương A. MÃ NGUỒN CÁC THÀNH PHẦN CHÍNH

A.1. Hàm main

```
1  //
2  // =====
3  // Tu Dien Anh-Viet (Phien ban OOP)
4  // Mon: Ky Thuat Lap Trinh
5  //
6  // File main: chi tao Application va goi run(). Toan bo logic
7  // duoc
8  // uy quyen cho cac doi tuong nghiep vu (Single Responsibility
9  // Principle).
10 //
11 // =====
12
13 #ifdef _WIN32
14 #include <windows.h>
15 #endif
16
17 #include <iostream>
18 #include "Application.h"
19 #include "Exceptions.h"
20
21 int main() {
22     #ifdef _WIN32
23         SetConsoleOutputCP(65001);
24         SetConsoleCP(65001);
25     #endif
26
27     try {
28         Application app;
29         app.initialize();
30         app.run();
31         app.shutdown();
32     } catch (const DictionaryException& e) {
33         std::cerr << "Loi he thong: " << e.what() << "\n";
34         return 1;
35     } catch (const std::exception& e) {
36         std::cerr << "Loi khong xac dinh: " << e.what() <<
37             "\n";
38         return 2;
39     }
40
41     return 0;
42 }
```


A.2. Facade: Application

A.2.1. Application.h

```

1  #ifndef APPLICATION_H
2  #define APPLICATION_H
3
4  #include "Dictionary.h"
5  #include "History.h"
6  #include "Favorites.h"
7  #include "IUserInterface.h"
8
9  //
10 // =====
11 // Application - Facade Pattern
12 //
13 // Dong vai tro mat tien: gom tat ca thanh phan cua he thong
14 // va expose
15 // mot API don gian (initialize + run + shutdown) cho main().
16 //
17 // Quan ly vong doi cua:
18 //   - Dictionary, History, Favorites (data layer)
19 //   - IUserInterface (presentation layer)
20 //
21 // Su dung composition: app KHONG inherit cac doi tuong tren,
22 // ma chua chung.
23 // Sau nay neu doi UI thanh GUI, chi can sua Application, main
24 // giu nguyen.
25 //
26 // =====
27
28 class Application {
29 private:
30     Dictionary dict_;
31     History history_;
32     Favorites favorites_;
33     IUserInterface* ui_; // pointer de cho phep polymorphism
34
35     static const int HISTORY_MAX_SIZE = 20;
36
37 public:
38     Application();
39     ~Application();
40
41     void initialize();
42     void run();
43     void shutdown();
44
45 private:
46     Application(const Application&);
47     Application& operator=(const Application&);

```

```

43 };
44
45 #endif // APPLICATION_H

```

A.2.2. Application.cpp

```

1  #include "Application.h"
2  #include "ConsoleUI.h"
3  #include "Exceptions.h"
4  #include <iostream>
5
6  static const std::string DICT_FILE = "data/dictionary.txt";
7  static const std::string HIST_FILE = "data/history.txt";
8  static const std::string FAV_FILE = "data/favorites.txt";
9
10 Application::Application()
11     : history_(HISTORY_MAX_SIZE), ui_(nullptr) {}
12
13 Application::~Application() {
14     delete ui_;
15 }
16
17 void Application::initialize() {
18     try {
19         dict_.loadFromFile(DICT_FILE);
20         std::cout << "Da tai " << dict_.size() << " tu tu \"\"
21                 << DICT_FILE << "\".\\n\"";
22     } catch (const FileIOException&) {
23         std::cout << "Khong tim thay \"\" << DICT_FILE
24                 << "\". Bat dau voi tu dien rong.\\n\"";
25     }
26
27     try { history_.loadFromFile(HIST_FILE); }
28     catch (const FileIOException&) {}
29     try { favorites_.loadFromFile(FAV_FILE); }
30     catch (const FileIOException&) {}
31
32     ui_ = new ConsoleUI(dict_, history_, favorites_);
33 }
34
35 void Application::run() {
36     if (ui_ == nullptr) {
37         throw DictionaryException("Application chua duoc
38                 initialize()");
39     }
40     ui_->run();
41 }
42
43 void Application::shutdown() {
44     // Cac doi tuong tu giai phong qua destructor (RAII)

```

A.3. Hệ thống Exception

```

1  #ifndef EXCEPTIONS_H
2  #define EXCEPTIONS_H
3
4  #include <exception>
5  #include <string>
6
7  //
8  // =====
9  // He thong Exception cho ung dung Tu Dien
10 //
11 // Cay ke thua:
12 //      std::exception
13 //      DictionaryException (base)
14 //      WordNotFoundException
15 //      DuplicateWordException
16 //      FileIOException
17 //      InvalidInputException
18 //
19 // =====
20
21 class DictionaryException : public std::exception {
22 protected:
23     std::string message_;
24 public:
25     explicit DictionaryException(const std::string& msg) :
26         message_(msg) {}
27     virtual ~DictionaryException() throw() {}
28     virtual const char* what() const throw() {
29         return message_.c_str();
30     }
31 };
32
33 class WordNotFoundException : public DictionaryException {
34 public:
35     explicit WordNotFoundException(const std::string& word)
36         : DictionaryException("Khong tim thay tu: \"" + word +
37                               "\"") {}
38 };
39
40 class DuplicateWordException : public DictionaryException {
41 public:
42     explicit DuplicateWordException(const std::string& word)
43         : DictionaryException("Tu da ton tai: \"" + word +
44                               "\"") {}
45 };

```

```

42 class FileIOException : public DictionaryException {
43 public:
44     explicit FileIOException(const std::string& filename)
45         : DictionaryException("Loi truy cap file: \"" +
46                               filename + "\"") {}
47 };
48
49 class InvalidInputException : public DictionaryException {
50 public:
51     explicit InvalidInputException(const std::string& msg)
52         : DictionaryException("Du lieu nhap khong hop le: " +
53                               msg) {}
54 };
55 #endif // EXCEPTIONS_H

```

A.4. Interface IPersistable

```

1  #ifndef I_PERSISTABLE_H
2  #define I_PERSISTABLE_H
3
4  #include <string>
5
6  //
7  // =====
8  // IPersistable - Interface cho cac doi tuong co the luu/tai
9  // tu file
10 //
11 // Abstract base class (pure virtual), khong the instantiate.
12 // Bat ky lop nao can persistence deu phai override 2 phuong
13 // thuc:
14 // - saveToFile(): ghi du lieu xuong file
15 // - loadFromFile(): doc du lieu tu file
16 //
17 // Hien tai duoc implement boi: Dictionary, History, Favorites.
18 //
19 // =====
20
21 class IPersistable {
22 public:
23     virtual ~IPersistable() {}
24
25     virtual void saveToFile(const std::string& filename) const
26         = 0;
27     virtual void loadFromFile(const std::string& filename) = 0;
28 };
29
30 #endif // I_PERSISTABLE_H

```

A.5. Strategy Pattern – ISearchStrategy

```

1  #ifndef I_SEARCH_STRATEGY_H
2  #define I_SEARCH_STRATEGY_H
3
4  #include <string>
5  #include "DynamicArray.h"
6  #include "Word.h"
7
8  //
9  // =====
10 // ISearchStrategy - Interface cho thuật toán tìm kiếm
11 // (Strategy Pattern)
12 //
13 // Cac implementation hien co:
14 //   - ExactSearch      : tìm chính xác (case-insensitive)
15 //   - ApproximateSearch : tìm gần đúng (Levenshtein distance)
16 //
17 // Co the de dang them moi:
18 //   - PrefixSearch     : tìm theo tiền tố (Trie)
19 //   - PhoneticSearch    : tìm theo phát âm (Soundex)
20 //
21 // =====
22
23 struct SearchResult {
24     int wordIndex;
25     int score;
26
27     SearchResult() : wordIndex(-1), score(0) {}
28     SearchResult(int idx, int s) : wordIndex(idx), score(s) {}
29 };
30
31 class ISearchStrategy {
32 public:
33     virtual ~ISearchStrategy() {}
34
35     virtual void search(const DynamicArray<Word>& words,
36                         const std::string& query,
37                         DynamicArray<SearchResult>& results)
38         const = 0;
39 };
40
41 #endif // I_SEARCH_STRATEGY_H

```

A.6. Dictionary.cpp (trích)

```

1  void Dictionary::loadFromFile(const std::string& filename) {
2      std::ifstream fin(filename.c_str());
3      if (!fin.is_open()) {

```

```
4         throw FileIOException(filename);
5     }
6
7     std::string line;
8     Word current;
9     bool inWord = false;
10
11     while (std::getline(fin, line)) {
12         StringUtils::trim(line);
13         if (line.empty()) continue;
14
15         if (line.length() >= 8 && line.substr(0, 8) ==
16             "@english") {
17             inWord = true;
18             current = Word();
19             current.setEnglish(line.substr(9));
20         } else if (line == "@end") {
21             if (!current.getEnglish().empty()) {
22                 try { addWord(current); }
23                 catch (const DuplicateWordException&) {}
24             }
25             inWord = false;
26         } else if (inWord) {
27             std::string val;
28             DynamicArray<std::string> parts;
29             if (line.length() >= 7 && line.substr(0, 7) ==
30                 "meaning") {
31                 val = line.substr(8);
32                 StringUtils::split(val, '|', parts);
33                 for (int i = 0; i < parts.size(); ++i)
34                     current.addMeaning(parts[i]);
35             } else if (line.length() >= 7 && line.substr(0, 7)
36                 == "example") {
37                 val = line.substr(8);
38                 StringUtils::split(val, '|', parts);
39                 for (int i = 0; i < parts.size(); ++i)
40                     current.addExample(parts[i]);
41             } else if (line.length() >= 7 && line.substr(0, 7)
42                 == "synonym") {
43                 val = line.substr(8);
44                 StringUtils::split(val, '|', parts);
45                 for (int i = 0; i < parts.size(); ++i)
46                     current.addSynonym(parts[i]);
47             }
48         }
49     }
50
51     if (inWord && !current.getEnglish().empty()) {
52         try { addWord(current); }
53         catch (const DuplicateWordException&) {}
54     }
```

```

48     fin.close();
49 }
50
51 void Dictionary::saveToFile(const std::string& filename) const
52 {
53     std::ofstream fout(filename.c_str());
54     if (!fout.is_open()) {
55         throw FileIOException(filename);
56     }
57
58     fout << "# File du lieu tu dien Anh-Viet\n";
59     fout << "# Moi tu mot block, bat dau @english va ket thuc
60         @end\n\n";
61
62     for (int i = 0; i < words_.size(); ++i) {
63         const Word& w = words_[i];
64         fout << "@english=" << w.getEnglish() << "\n";
65         writeField(fout, "meaning", w.getMeanings());
66         writeField(fout, "example", w.getExamples());
67         writeField(fout, "synonym", w.getSynonyms());
68         fout << "@end\n\n";
69     }
70     fout.close();
71 }

```

A.7. History.cpp (Rule of Three)

```

1 History::History(int maxSize)
2     : head_(nullptr), count_(0), maxSize_(maxSize) {}
3
4 History::History(const History& other)
5     : head_(nullptr), count_(0), maxSize_(other.maxSize_) {
6     copyFrom(other);
7 }
8
9 History& History::operator=(const History& other) {
10     if (this == &other) return *this;
11     freeAll();
12     maxSize_ = other.maxSize_;
13     copyFrom(other);
14     return *this;
15 }
16
17 History::~History() {
18     freeAll();
19 }
20
21 void History::add(const std::string& word) {
22     // 1) Neu da co, xoa ban cu
23     Node* prev = nullptr;

```

```

24     Node* cur = head_;
25     while (cur) {
26         if (cur->word == word) {
27             if (prev == nullptr) head_ = cur->next;
28             else prev->next = cur->next;
29             delete cur;
30             --count_;
31             break;
32         }
33         prev = cur;
34         cur = cur->next;
35     }
36
37     // 2) Chen vao dau
38     Node* newNode = new Node(word);
39     newNode->next = head_;
40     head_ = newNode;
41     ++count_;
42
43     // 3) Cat duoi neu vuot gioi han
44     if (count_ > maxSize_) {
45         Node* p = head_;
46         while (p->next && p->next->next) p = p->next;
47         delete p->next;
48         p->next = nullptr;
49         --count_;
50     }
51 }
52
53 void History::saveToFile(const std::string& filename) const {
54     std::ofstream fout(filename.c_str());
55     if (!fout.is_open()) {
56         throw FileIOException(filename);
57     }
58     Node* cur = head_;
59     while (cur) {
60         fout << cur->word << "\n";
61         cur = cur->next;
62     }
63     fout.close();
64 }

```